

# Spring Boot

## Complete Topic Guide

0 to 7+ Years Experience Roadmap  
Fresher • Junior • Mid-Level • Senior • Architect

---

A structured checklist of Spring Boot concepts, mapped to experience level, for learning, revision, and interview preparation.

# Table of Contents

1. How to Use This Guide
2. 0–1 Years — Fresher / Trainee Level
3. 1–2 Years — Junior Developer
4. 2–4 Years — Mid-Level Developer
5. 4–6 Years — Senior Developer
6. 6–7+ Years — Lead / Architect Level
7. Spring Boot Ecosystem Cheat Sheet
8. Commonly Asked Interview Questions (Topic-wise)
9. Recommended Practice Projects

## 1. How to Use This Guide

This guide breaks down Spring Boot topics by experience band, similar to how interviewers and companies typically scope expectations. Topics are cumulative — each level assumes solid command of everything in the levels before it. Use it as a learning roadmap if you're starting out, or as a revision/interview-prep checklist if you already have experience.

Tip: Don't just read the topic names — for each one, be able to explain **what it is**, **why it's used**, and **a scenario where you applied or would apply it**.

## 2. 0–1 Years — Fresher / Trainee Level

Focus: Core Java + Spring fundamentals + building a basic REST API.

### Java & OOP Prerequisites

- Core Java 8+: OOP concepts, collections, exception handling, streams, lambda expressions
- Interfaces vs abstract classes, generics basics
- Maven or Gradle basics — project structure, dependencies, build lifecycle

### Spring Core Concepts

- What is Spring Framework vs Spring Boot
- Inversion of Control (IoC) and Dependency Injection (DI)
- Spring Beans, Bean lifecycle, Application Context
- @Component, @Service, @Repository, @Controller annotations
- Constructor vs Setter vs Field injection
- @Autowired and @Qualifier

### Spring Boot Basics

- Spring Initializr — creating a project
- @SpringBootApplication annotation and auto-configuration
- application.properties / application.yml basics
- Spring Boot starters (spring-boot-starter-web, starter-data-jpa, etc.)
- Embedded servers (Tomcat/Jetty) and how Spring Boot runs standalone

### Building REST APIs

- @RestController, @RequestMapping, @GetMapping/@PostMapping/@PutMapping/@DeleteMapping
- Path variables (@PathVariable) and request parameters (@RequestParam)
- Request body binding with @RequestBody, response with ResponseEntity
- Basic HTTP status codes and their meaning
- Simple validation using @Valid and Bean Validation (javax/jakarta.validation)

### Database Basics

- Spring Data JPA basics — Entity, @Id, @GeneratedValue
- CrudRepository / JpaRepository
- Basic H2 / MySQL configuration
- Simple CRUD operations end-to-end

### Tools

- Postman for API testing
- Git basics — clone, commit, push, pull, branch
- IDE usage (IntelliJ / Eclipse / VS Code) and debugging basics

## 3. 1–2 Years — Junior Developer

Focus: Writing production-quality APIs, layered architecture, and testing.

### Application Architecture

- Layered architecture — Controller, Service, Repository, DTO
- DTO vs Entity, and why not to expose entities directly
- Model mapping tools (MapStruct, ModelMapper)
- Exception handling — @ExceptionHandler, @ControllerAdvice, custom exceptions

### Spring Data JPA (Deeper)

- Entity relationships — @OneToMany, @ManyToOne, @ManyToMany, @OneToOne
- Fetch types — LAZY vs EAGER
- JPQL and native queries (@Query)
- Pagination and sorting (Pageable, Sort)
- Auditing basics (@CreatedDate, @LastModifiedDate)

### Configuration & Profiles

- Spring Profiles (@Profile, application-{profile}.yml)
- Externalized configuration and @ConfigurationProperties
- @Value annotation usage

### Testing

- JUnit 5 basics — @Test, assertions, lifecycle annotations
- Mockito — mocking dependencies, verifying interactions
- @SpringBootTest, @WebMvcTest, @DataJpaTest
- MockMvc for controller-layer testing

### Logging & Basic Ops

- SLF4J + Logback basics
- Log levels and where to place log statements
- Basic Actuator endpoints (/health, /info)

### Version Control & Collaboration

- Git branching strategies (feature branch, git flow basics)
- Code review etiquette
- Writing basic API documentation (Swagger / OpenAPI basics)

## 4. 2–4 Years — Mid-Level Developer

Focus: Security, transactions, messaging, microservices fundamentals, and performance basics.

### Spring Security

- Authentication vs Authorization
- Spring Security filter chain
- Form login, Basic Auth, and stateless JWT-based auth
- Role-based and method-level security (@PreAuthorize, @Secured)
- CORS configuration and CSRF basics
- OAuth2 / OpenID Connect fundamentals

### Transactions

- @Transactional — propagation types and isolation levels
- Transaction rollback rules
- Common pitfalls: self-invocation, proxy limitations

### Caching

- Spring Cache abstraction — @Cacheable, @CacheEvict, @CachePut
- Integrating Redis / Ehcache as a cache provider

### Messaging & Async

- @Async and thread pool configuration
- Spring Kafka or RabbitMQ basics — producer/consumer
- Event-driven basics — ApplicationEventPublisher, @EventListener

### Microservices Fundamentals

- Monolith vs microservices trade-offs
- Service discovery (Eureka) and API Gateway concepts
- Inter-service communication — RestTemplate vs WebClient vs Feign
- Circuit breaker basics (Resilience4j / Hystrix concepts)
- Centralized configuration (Spring Cloud Config) basics

### Performance & Reliability

- Connection pooling (HikariCP) tuning basics
- N+1 query problem and how to avoid it
- Basic JVM tuning awareness (heap, GC types at a high level)
- Rate limiting and retry basics

### Testing (Deeper)

- Integration testing with Testcontainers
- Contract testing basics (e.g., Spring Cloud Contract)
- Code coverage tools (JaCoCo)

## 5. 4–6 Years — Senior Developer

Focus: System design, scalability, observability, and leading technical decisions.

### Advanced Microservices

- Saga pattern — choreography vs orchestration
- Distributed transactions and eventual consistency
- API versioning strategies
- Bulkhead, circuit breaker, and rate limiter patterns in depth (Resilience4j)
- Service mesh concepts (Istio) at a conceptual level

### Observability

- Distributed tracing (Sleuth/Zipkin, OpenTelemetry)
- Centralized logging (ELK / Loki stack) integration
- Metrics with Micrometer + Prometheus + Grafana
- Custom Actuator health indicators and endpoints

### Advanced Spring Internals

- Bean lifecycle callbacks — `@PostConstruct`, `@PreDestroy`, `BeanPostProcessor`
- How Spring Boot auto-configuration works internally (`@Conditional`, `spring.factories` / `AutoConfiguration.imports`)
- Custom starter creation
- AOP — `@Aspect`, pointcuts, advice types, and real use cases (logging, auditing)

### Data & Scaling

- Database sharding and partitioning concepts
- Read/write replica strategies
- Optimistic vs pessimistic locking
- CQRS pattern basics
- Caching strategies at scale (cache invalidation, write-through/write-behind)

### Security (Advanced)

- Token refresh strategies, JWT best practices, key rotation
- Secrets management (Vault, AWS Secrets Manager)
- Securing service-to-service communication (mTLS basics)

### DevOps & Deployment

- Dockerizing Spring Boot applications
- Kubernetes basics — deployments, services, config maps, secrets
- CI/CD pipeline design (build, test, deploy stages)
- Blue-green and canary deployment concepts

### System Design

- Designing scalable REST/microservice architectures

- Trade-off discussions: consistency vs availability (CAP theorem)
- Load balancing strategies
- Idempotency in API design

## 6. 6–7+ Years — Lead / Architect Level

Focus: Architecture ownership, cross-cutting technical strategy, mentorship, and org-level decisions.

### Architecture & Strategy

- Choosing monolith vs modular monolith vs microservices for a given org context
- Designing multi-region, highly available systems
- Domain-Driven Design (DDD) — bounded contexts, aggregates, ubiquitous language
- Event sourcing at an architectural level
- API Gateway and BFF (Backend-for-Frontend) pattern design

### Reliability Engineering

- Designing for failure — chaos engineering principles
- SLA, SLO, SLI definitions and how architecture supports them
- Capacity planning and cost-performance trade-offs
- Disaster recovery and multi-region failover design

### Platform & Governance

- Defining coding standards, architectural decision records (ADRs)
- Building internal frameworks/libraries and shared platform services
- Dependency and version governance across services
- Technical debt assessment and prioritization frameworks

### Performance at Scale

- JVM tuning in depth — GC algorithms (G1, ZGC), heap sizing, profiling tools (JFR, async-profiler)
- Load testing strategy (JMeter/Gatling) and interpreting results
- Designing for horizontal scalability under real production load

### Leadership & Process

- Mentoring and conducting design/code reviews at scale
- Driving technology choices and vendor/tool evaluations
- Estimation, roadmap planning, and stakeholder communication
- Incident management — postmortems and root cause analysis

## 7. Spring Boot Ecosystem Cheat Sheet

Quick reference of common Spring Boot / Spring Cloud modules and what they're used for.

| Module                      | Purpose                                        |
|-----------------------------|------------------------------------------------|
| Spring Web (MVC)            | Building REST APIs / web applications          |
| Spring Data JPA             | ORM-based database access via repositories     |
| Spring Security             | Authentication & authorization                 |
| Spring Cloud Config         | Centralized externalized configuration         |
| Spring Cloud Netflix Eureka | Service discovery/registry                     |
| Spring Cloud Gateway        | API gateway / routing                          |
| Spring Cloud OpenFeign      | Declarative REST client for service calls      |
| Resilience4j                | Circuit breaker, retry, rate limiter, bulkhead |
| Spring Kafka / AMQP         | Messaging with Kafka / RabbitMQ                |
| Spring Batch                | Batch processing of large datasets             |
| Spring Actuator             | Production-ready monitoring endpoints          |
| Spring Cache                | Caching abstraction (Redis, Ehcache, Caffeine) |
| Spring WebFlux              | Reactive, non-blocking web applications        |
| Spring Boot Test            | Testing support (unit + integration)           |
| Micrometer                  | Application metrics facade (Prometheus, etc.)  |

## 8. Commonly Asked Interview Questions (Topic-wise)

### Core / Fresher-Junior

- What is the difference between Spring and Spring Boot?
- Explain Dependency Injection with an example.
- What is auto-configuration and how does Spring Boot achieve it?
- Difference between @Component, @Service, and @Repository.
- How do you handle exceptions globally in a Spring Boot app?

### Mid-Level

- Explain @Transactional propagation and isolation levels.
- How does Spring Security's filter chain work?
- What is the N+1 query problem and how do you fix it?
- Difference between RestTemplate, WebClient, and Feign client.
- How would you implement caching in a Spring Boot service?

### Senior / Architect

- How would you design a resilient microservice architecture with retries, circuit breakers, and fallbacks?
- Explain the Saga pattern and when you'd choose orchestration over choreography.
- How do you approach zero-downtime deployments for a Spring Boot service?
- How would you debug a memory leak or high GC pause time in production?
- How do you design multi-tenant Spring Boot applications?

## 9. Recommended Practice Projects

### Beginner Projects

- Simple CRUD REST API (e.g., Book/Student management system)
- To-do list app with H2/MySQL and basic validation

### Intermediate Projects

- E-commerce backend with JWT auth, roles, and order/payment flow
- Blog platform with pagination, caching, and file uploads

### Advanced Projects

- Microservices-based food delivery / ride-booking system with Eureka, Gateway, and Kafka
- Distributed order-processing system with Saga pattern and Resilience4j
- Multi-tenant SaaS backend with observability (metrics, tracing, centralized logs)

---

*Good luck with your Spring Boot learning and interview preparation!*